

FrameWatch

Asymmetric News Framing Detector with Cross-Lingual Drift Analysis

Cloud Architecture & Design Document · Microsoft Azure

1. Project Overview

FrameWatch is a cloud-native Azure service that detects asymmetric linguistic framing in news statements. The core insight is that factually identical information can be presented in ways that prime readers to feel sympathy for one side over another — through verb choice, qualifier asymmetry, entity ordering, and emotional word imbalance. FrameWatch extends this analysis across languages: using Azure Translator, it detects how framing asymmetry shifts when the same statement is translated, revealing a second-order bias that is invisible to monolingual tools.

[Classic example — framing asymmetry in English:](#)

"20 killed in Country B, 15 dead in Country C." — The verb killed implies an active perpetrator; dead is neutral/passive. The facts are identical; the emotional loading is not.

[Cross-lingual framing drift example:](#)

Arabic original: "15 martyrs fell in Country C, 20 were killed in Country B." — The word martyr carries strong sympathetic connotation absent in English. After translation, this emotional loading is neutralised. FrameWatch detects this as a Cross-Lingual Framing Drift Score (CFDS), quantifying how much the asymmetry changes across languages.

2. Problem Statement

Existing sentiment tools treat a document as a single polarity unit. They cannot detect when one named entity within the same sentence is framed sympathetically while another is framed adversarially — nor can they detect when translation itself introduces or erases that asymmetry. FrameWatch addresses both gaps: it produces a per-entity Framing Asymmetry Score (FAS) and a Cross-Lingual Framing Drift Score (CFDS) for any multilingual news input.

3. Azure Service Architecture

FrameWatch uses 8 Azure services, satisfying all course requirements:

Course Requirement	Azure Service	Role in FrameWatch
Storage	Azure Blob Storage	Stores raw input text (original + translated) and analysis history
Database	Azure Cosmos DB (NoSQL)	Persists structured FAS, CFDS results, entity scores, session logs
Infrastructure	Azure App Service	Hosts the HTML/CSS/JS frontend web application
ML / AI — #1	Azure AI Language Service	Sentiment, Opinion Mining, NER, Key Phrase Extraction
ML / AI — #2	Azure Translator	Multilingual translation + language detection for cross-lingual drift analysis
Novel Service 1	Azure Functions	Serverless compute for FAS scoring logic and CFDS calculation

Novel Service 2	Azure Cognitive Search	Indexes past analyses; search by entity, topic, date, drift score
Novel Service 3	Azure Logic Apps	Event-driven pipeline: Blob upload → trigger full analysis chain

4. System Architecture & Data Flow

4.1 Architecture Layers

Layer	Components	Description
Presentation	Azure App Service + HTML/CSS/JS	User submits news text; results displayed as framing dashboard
Orchestration	Azure Logic Apps	Monitors Blob for new uploads; triggers Function pipeline automatically
Compute	Azure Functions (3 functions)	TranslateAndExtract → FramingScore → SaveResults
AI/ML	Azure AI Language Service	Opinion Mining + NER run against each uploaded statement
Storage	Azure Blob Storage	Raw text archive; immutable input store
Database	Azure Cosmos DB	Structured JSON results: FAS scores, entity pairs, timestamps
Discovery	Azure Cognitive Search	Full-text + vector index over all stored analyses

4.2 Pipeline Data Flow

- 1. User pastes a news statement (any language) into the App Service frontend.
- 2. Frontend uploads raw text to Azure Blob Storage (container: raw-inputs).
- 3. Azure Logic App detects the Blob upload event and triggers the Azure Function pipeline.
- 4. Function 1 — TranslateAndDetect: calls Azure Translator to translate text to English and detect source language. Both original and translated versions are saved to Blob Storage.
- 5. Function 2 — ExtractAndScore (Original): calls Azure AI Language NER + Opinion Mining on the original-language text to compute FAS-original.
- 6. Function 3 — ExtractAndScore (Translated): calls Azure AI Language NER + Opinion Mining on the English translation to compute FAS-translated.
- 7. Function 4 — ComputeCFDS: calculates Cross-Lingual Framing Drift Score = $|FAS-original - FAS-translated|$. High drift signals that translation has materially shifted the framing asymmetry.

- 8. Function 5 — PersistResults: writes the full structured JSON (FAS, CFDS, entity scores, signals) to Cosmos DB and pushes to Cognitive Search index.
- 9. Frontend polls Cosmos DB via API and renders the framing dashboard with both FAS and CFDS panels.

5. Core Analytical Modules

5.1 Framing Asymmetry Score (FAS) — Entity-Level Detection

The FAS quantifies how much more sympathetically Entity A is framed compared to Entity B within the same statement. Score range: -1.0 to +1.0.

$$\text{FAS} = \text{SentimentScore}(\text{Entity A}) - \text{SentimentScore}(\text{Entity B})$$

FAS interpretation:

FAS Range	Interpretation	Example
+0.5 to +1.0	Entity A strongly favoured	"killed" vs "died" — A is victim-framed
+0.1 to +0.5	Mild framing in favour of A	Qualifier asymmetry: "alleged" applied only to B
-0.1 to +0.1	Balanced framing	Both entities described with equal tone
-0.5 to -0.1	Mild framing in favour of B	B named first; A given passive verb
-1.0 to -0.5	Entity B strongly favoured	Alarmist language directed only at A

5.2 Cross-Lingual Framing Drift Score (CFDS) — The Translator Module

The CFDS is FrameWatch's second core metric, enabled by Azure Translator. It measures how much the framing asymmetry between two entities changes after the statement is translated into English. A high CFDS reveals that translation itself is a source of bias — either introducing framing that was absent in the original, or erasing framing that was present.

$$\text{CFDS} = | \text{FAS}(\text{original language}) - \text{FAS}(\text{translated English}) |$$

CFDS Range	Interpretation	Real-World Implication
0.00 – 0.10	Stable framing across languages	Translation preserves the original asymmetry faithfully
0.10 – 0.30	Moderate drift	Some framing nuance is lost or gained in translation
0.30 – 0.60	Significant drift	Translation materially changes which side appears favoured
0.60 – 1.00	Extreme drift — translation-induced bias	A neutral original becomes biased (or vice versa) after translation

Azure Translator API calls made by FrameWatch:

Call	Endpoint	Parameters	Output
Translate to English	api.cognitive.microsofttranslator.com/translate	to=en, from=auto-detect	Translated text + detected source language
Language Detection	api.cognitive.microsofttranslator.com/detect	Text body	Language code + confidence score

5.3 Five Framing Signal Detectors (inputs to FAS)

Verb Agency: "killed/slaughtered/massacred" (active, implies perpetrator) vs "died/fell/were lost" (passive, no blame). [Method: Azure Opinion Mining sentiment on verb targets]

Qualifier Asymmetry: "Alleged/suspected/claimed" applied to one entity but not the other. [Method: Key Phrase Extraction + rule-based qualifier detection]

Ordering Bias: The entity named first in a conflict statement receives primacy — readers anchor on it. [Method: Position index of NER entity spans]

Emotional Word Load: Count of emotionally charged words (massacre, devastation, horror) associated with each entity. [Method: Lexicon lookup on Opinion Mining aspect-level text]

Numeric Context: Same numbers but one is given more emotional surrounding context ("only 15" vs "as many as 20"). [Method: Dependency pattern matching around cardinal entities]

5.3 Azure AI Language — Opinion Mining Detail

Opinion Mining (a feature of Azure AI Language Sentiment Analysis) returns aspect-level sentiment: for each opinion target in a sentence it returns the target text, the opinion text, and a positive/negative/neutral confidence score. FrameWatch uses this to assign sentiment directly to each named entity rather than to the document as a whole.

API Call	Endpoint	Output used by FrameWatch
SentimentAnalysis (opinionMining=true)	/language/:analyze-text	Per-entity opinion targets + sentiment scores
EntityRecognition	/language/:analyze-text	Named entity spans + categories (Person/Org/Location)
KeyPhraseExtraction	/language/:analyze-text	Dominant themes; qualifier detection input

6. Azure Function — Framing Score Logic (Pseudocode)

The core business logic runs inside an Azure Function (Python runtime). It is triggered by the Logic App after NER results arrive from Azure AI Language.

```
def compute_fas(ner_entities, opinion_mining_result):
    entity_scores = {}
    for opinion in opinion_mining_result.opinions:
        target = find_closest_entity(opinion.target, ner_entities)
        if target:
            score = opinion.positive_score - opinion.negative_score
            entity_scores[target] = entity_scores.get(target, []) + [score]

    avg_scores = {e: mean(scores) for e, scores in entity_scores.items()}

    # Ordering bias: penalise entity mentioned later
    sorted_by_position = sorted(ner_entities, key=lambda e: e.offset)
    for i, ent in enumerate(sorted_by_position):
        avg_scores[ent.text] = avg_scores.get(ent.text, 0) - (i * 0.05)

    # Qualifier asymmetry
    QUALIFIERS = ["alleged", "suspected", "claimed", "purported", "so-called"]
    for ent in ner_entities:
        if any(q in get_surrounding_text(ent) for q in QUALIFIERS):
            avg_scores[ent.text] = avg_scores.get(ent.text, 0) - 0.15

    entities = list(avg_scores.keys())
    if len(entities) >= 2:
        fas = avg_scores[entities[0]] - avg_scores[entities[1]]
        return {"fas": round(fas, 3), "entity_a": entities[0],
                "entity_b": entities[1], "scores": avg_scores}
    return {"fas": 0, "note": "insufficient entities detected"}
```

7. Data Storage Design

7.1 Cosmos DB Document Schema

```
{
  "id": "uuid-v4",
  "timestamp": "2025-03-05T12:00:00Z",
  "input_text": "20 qatil fi al-dawla B, 15 mata fi al-dawla C",
  "source_language": "ar",
  "translated_text": "20 killed in Country B, 15 dead in Country C",
  "content_type": "news",
  "fas_original": 0.71,
  "fas_translated": 0.62,
  "cfds": 0.09,
  "entity_a": { "name": "Country B", "type": "Location",
    "score_original": 0.36, "score_translated": 0.31 },
  "entity_b": { "name": "Country C", "type": "Location",
    "score_original": -0.35, "score_translated": -0.31 },
  "signals": {
    "verb_agency": 0.70,
    "qualifier_asymmetry": 0.00,
    "ordering_bias": 0.05,
    "emotional_load": 0.45,
    "numeric_context": 0.30
  },
  "overall_sentiment": "negative",
  "key_phrases": ["conflict casualties", "policy response"],
  "blob_ref": "raw-inputs/uuid-v4.txt"
}
```

7.2 Blob Storage Structure

Container	Contents	Access Tier
raw-inputs	Original text files (.txt), one per submission	Hot
translated-inputs	English-translated versions of each submission	Hot
analysis-outputs	Full JSON analysis results archive	Cool
logs	Logic App execution logs	Cool

8. Azure Deployment Steps

Step	Service	Action
Step 1	Resource Group	Create a Resource Group (e.g. framewatch-rg) in your chosen region (East US recommended).
Step 2	Azure AI Language	Create Language resource. Enable: Sentiment Analysis, Opinion Mining, NER, Key Phrase Extraction. Copy Key + Endpoint.
Step 3	Azure Translator	Create Translator resource (same region). Copy Key + Region. Supports 100+ languages auto-detected at runtime.
Step 4	Azure Blob Storage	Create Storage Account. Add containers: raw-inputs, translated-inputs, analysis-outputs, logs. Note connection string.
Step 5	Azure Cosmos DB	Create Cosmos DB (NoSQL API). Create database framewatch-db, container analyses, partition key / content_type.
Step 6	Azure Functions	Create Function App (Python 3.11). Deploy 5 functions: TranslateAndDetect, ScoreOriginal, ScoreTranslated, ComputeCFDS, PersistResults. Add all keys as Application Settings.
Step 7	Azure Logic Apps	Create Logic App. Trigger: Azure Blob Storage — When a blob is added. Action chain: call each Function in sequence.
Step 8	Azure Cognitive Search	Create Search Service. Create index framewatch-index with fields: fas_original, fas_translated, cfds, entity_a, entity_b, source_language. Configure indexer from Cosmos DB.
Step 9	Azure App Service	Create App Service (Free F1 tier). Deploy frontend HTML/CSS/JS. Route Language + Translator calls through Functions to handle CORS.

9. Use Cases & Innovation Value

9.1 Use Cases

- Conflict journalism analysis — detect which side of a war is systematically victim-framed across outlets.
- Political speech auditing — compare how two candidates are described in the same article.
- Trade dispute reporting — identify asymmetric framing of countries in economic news.
- Media literacy education — show students concrete examples of framing with scored evidence.
- NGO monitoring — track framing of humanitarian crises across different news sources over time.

9.2 What Makes This Novel

Generic sentiment analysis assigns one polarity to a whole document. FrameWatch operates at the entity-pair level: it compares the sentiment directed at Entity A vs Entity B within the same statement. This mirrors how human readers actually experience framing — not as an overall tone but as a differential treatment of actors in the same story. No standard Azure demo or tutorial covers this use case.

10. Summary Service Checklist

✓	Azure Service	Requirement Met
✓	Azure Blob Storage	Storage
✓	Azure Cosmos DB	Database
✓	Azure App Service	Infrastructure
✓	Azure AI Language Service	ML / AI — #1 (FAS scoring via Opinion Mining + NER)
✓	Azure Translator	ML / AI — #2 (Cross-Lingual Framing Drift via CFDS)
✓	Azure Functions	Novel Service #1 (Serverless framing score compute)
✓	Azure Cognitive Search	Novel Service #2 (Analysis discovery & indexing)
✓	Azure Logic Apps	Novel Service #3 (Event-driven pipeline orchestration)